01

Why your build broke at 80%

What happens in the context window when the AI starts undoing your features. The structural reason, not the vibes.

SUBJECT	CONTEXT WINDOWS · DRIFT	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	FREE CHAPTER OF TEN
TIME	A TEN-MINUTE READ	REV	2026.08

§ 01.1

The moment it turns

You asked for something small. Move a button. Fix a date format. The diff was four lines. It compiled. You shipped it.

Then you clicked around. Uploads are broken. Uploads. The feature you finished Tuesday. The one that already worked.

So you tell the AI to fix uploads. It fixes uploads. Now the date format is wrong again.

Somewhere around the third loop, everyone has the same thought: this thing has turned on me.

It has not. It is doing exactly what it was built to do, with exactly the information it has. The problem is structural, which is the good news. Structural problems can be engineered around, and engineering around this one is what separates the builds that ship from the graveyard of demos.

This chapter is the mechanism, in plain terms, and then five habits that stop the bleeding tonight. The other nine chapters build the cure.

FIELD NOTE

Everything in this manual comes from builds I shipped. Mostly Ordani: a HIPAA-compliant SaaS I built alone with Claude Code and Cursor, that hundreds of birth workers pay for. Same stack you're using. Same wall.

§ 01.2

What the AI actually remembers

Every AI coding tool, whatever the logo, works inside a **context window**.

CONTEXT WINDOW

> The fixed span of text the model can consider at one time: your instructions, the files it read, the diffs it wrote, the errors it saw. Everything outside the window does not exist for it. Not "harder to recall." Does not exist.

The window is not memory. It is a transcript with a hard length limit. In 2026 the big models hold somewhere between two hundred thousand and a million tokens, and a working session eats that fast: every file the tool opens lands in the transcript, along with every diff, every stack trace, every "here's what I found."

When the transcript fills, one of two things happens. The oldest content falls off the end, or the tool compresses it into a summary that keeps what looked important and quietly drops what looked incidental. "Looked" is the word doing all the damage.

Rough math: a token is about three-quarters of a word. Two hundred thousand tokens is a long novel. A session that reads files, writes diffs, and chases errors gets through a novel quickly.

And between sessions, almost nothing survives. Yes, your tool has a resume flag, and maybe a memory feature. Resume restores one conversation thread. Memory keeps what the tool guessed was important. Neither is a system you control. By default, tomorrow's session starts with three things: your codebase, your prompt, and no memory of how the codebase got this way.

§ 01.3

Features are more than their code

Here is the part almost nobody tells you. It is the crux of this whole manual.

A working feature is not just code. It is code plus the rules that make the code correct. "Uploads must stream to storage, never buffer, because a 500MB video crushes the server." "Dates format on the server, because the client's timezone lies." Engineers call a rule like this an invariant: something that must stay true no matter what changes around it. The code shows what it does. The reason it must be done that strange way lived in exactly one place: the conversation where you and the AI worked it out.

That conversation is gone.

So weeks later, a fresh session opens your upload file while doing something unrelated. It sees an oddly complicated streaming setup where a simple buffer would do. It has never heard of the 500MB video. Simplifying that code is the correct move given everything it can see. It is not sabotaging your feature. It is tidying code whose reason for existing is written down nowhere on earth.

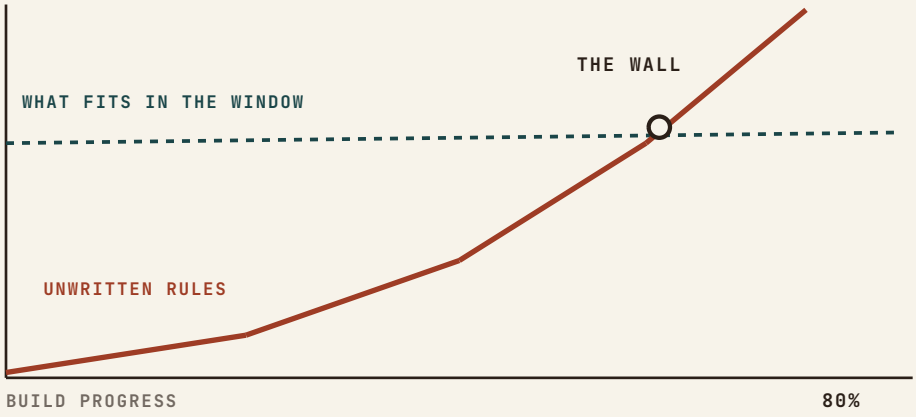
Code says what. Only the transcript knew why. And the transcript is gone.

Now multiply by every feature you have shipped. That is the true shape of your app at 80%: held together by dozens of invisible rules, inspected by a tool that can only see the code.

§ 01.4

Why it hits at 80% and not sooner

The wall is not one failure. It is two lines crossing.



Your codebase outgrows the window. In week one the whole project fits in context. The AI is briefly all-knowing, and it feels like magic. The magic is real, and it is temporary. A mid-sized app runs a few hundred files, several hundred thousand tokens, and past that the AI is no longer reading your codebase. It is sampling it.

Your unwritten rules pile up. Every feature that works adds constraints to the pile: the auth rule, the streaming rule, the timezone rule, the webhook-ordering rule. At 20% built you carry three of these in your head. At 80% you carry sixty, they interact, and not one is written down.

Your sessions multiply. By 80% you are dozens of sessions deep. Each started amnesiac. Each rebuilt its own partial picture of the app from whatever files it happened to open. Session forty's mental model disagrees with session twelve's, and both wrote code you are still running.

The wall is the point where the invisible rules outnumber what fits in the window. It is not a talent problem. It is arithmetic, and arithmetic can be beaten with a system.

§ 01.5

The regression loop

Cross the wall without knowing it, and the same spiral catches nearly everyone:

1. You ask for change B. The AI delivers it, quietly violating rule A, which was nowhere in its view.
2. You spot A broken and ask for a fix. The fix trips rule C, or re-breaks B.
3. The fixes keep compiling, so you stop reading the diffs. Drift now compounds silently.
4. Frustration peaks, and the thought arrives: maybe I should just have it rebuild the whole thing clean.

> The rewrite is the wall wearing a costume. A rebuild trades bugs you know for bugs you do not, and resets your pile of hard-won rules to zero. That pile was the only thing you had earned. Rebuilds stall at their own 60%, for exactly the reasons the original stalled at 80.

FIELD NOTE

The break rarely announces itself. The first symptom is usually a regression in something you had stopped watching, days after the session that caused it.

§ 01.6

Two entries from my build log

I hit this wall in the same month I am writing this, on my own consulting site, with a long engineering career behind me. The wall does not care about your resume. Both entries below are real, with dates.

FROM THE BUILD LOG

ENTRY · 2026-08-11

The same bug, twice

My site's framework has a quirk: in certain conditions the renderer eats the space after bold text, so "$20M+ in client revenue" shipped to production as "$20M+in client revenue." A session found it, fixed it, and wrote the lesson in the project's docs. Case closed.

Weeks later, a different session reintroduced the exact same bug in new copy. It had never read the lesson. Writing it down was not enough, because prose only works on whoever reads it, and the next session reads nothing you don't put in front of it.

What finally stuck was a one-line check in the repo, run before every release. Simplified, it looks like this:

```
grep -rn "+in client" app/ && echo "SPACE BUG IS BACK" && exit 1
```

In plain English: search every file for the broken pattern, and if it appears anywhere, shout and refuse to ship. The bug never shipped again. Rules a machine can run beat rules a reader must remember. That idea repeats through every chapter of this manual.

FROM THE BUILD LOG

ENTRY · 2026-08-31

The demo that lied for weeks

That same site had three lead forms: contact, a sample-chapter signup, a beta waitlist. All three worked flawlessly in the demo. In production, for weeks, every submission fell into a server log nobody reads, because one environment variable, the email key, was never installed on the live host.

No error. No bounce. The page told every visitor "Got it."

I found out this morning, only because I tested a new feature end to end on the live site and that test failed loudly enough to make me look. Two lessons, each with its own chapter later: production is a different machine than the demo. And "it works" is a claim about the path you actually tested, never about the code you wrote.

§ 01.7

What does not fix it

Four false exits. I have paid for all four.

A smarter model. Smarter per glance, same amnesia. Drift is not caused by low intelligence. It is caused by what is out of view.

A bigger window. A million tokens digests a bigger codebase. It does not conjure the sixty reasons-why that were never written anywhere a window could hold.

Telling it to remember. "Always keep uploads streaming" pins the rule to one transcript. Politeness is not persistence. The next session never heard you.

Pasting everything, every time. Stuffing the repo into each prompt burns the window on what while still holding zero why, and crowds out the room the model needs to think.

§ 01.8

What works: memory the window cannot lose

The move that changes everything is small: stop treating the conversation as storage.

Anything that must survive the session goes in a file, in the repo, because files are the only thing every future session is guaranteed to see. A rules file has a second advantage: when the tool compresses your conversation, the transcript gets summarized, but the rules file gets re-read, whole, every time. Your role quietly changes at the wall. You stop being the person who asks for features, and become the keeper of the memory the tool does not have. That is not a demotion. It is the actual job, and the people who accept it are the ones who ship.

PRE-FLIGHT · FIVE HABITS — RUN TONIGHT

- [ ] **Write the invariant list.** One file in the repo root, named whatever your tool reads on its own (see the file card below). One line per rule-with-a-reason: “Uploads stream, never buffer: 500MB videos kill the function.” Thirty minutes, tonight.

- [ ] **Point every session at it first.** “Read the invariants file before you touch anything.” Ten seconds that stands in for every session that came before this one.

- [ ] **Read every diff before accepting.** The moment you stop reading is the moment drift starts compounding. If a diff touches a file your change had no business touching, stop and ask why.

- [ ] **One change per commit, committed when it works.** Small commits are restore points. Rolling back beats an apology prompt, every single time.

- [ ] **Caught the AI breaking a rule? Write the rule down, same day.** Fix the code, add the rule to the file, and where you can, turn it into a check a machine runs. Nothing in this manual pays back more per minute.

So you don’t have to guess what habit one produces, here is the shape of the file, three rules in. Yours will grow past twenty.

CLAUDE.md — invariants

```
# Rules that must survive every session. Read
before any change.
• Uploads STREAM to storage, never buffer. 500MB
  videos kill the server.
• Dates format on the SERVER. The client's timezone
  lies.
• Never hand-edit files in /generated. Regenerate
  them or nothing.
```

These five stop the bleeding. They are triage, not the cure. The cure is a single page the AI re-reads at the start of every session, carrying your architecture, your constraints, and your definition of done. Building that page is Chapter 2, and it is the reason this manual exists.

Tool note: Claude Code reads CLAUDE.md automatically. Cursor reads the .mdc rule files in .cursor/rules. Codex and several others read AGENTS.md. Find the file your tool reads first. Same idea, same payoff.

The wall is not proof you can’t do this. It is the point where the tool’s memory ran out and yours has to take over: on paper, in the repo, where every future session finds it.

WHO WROTE THIS

I'm Micah Jones. I sold enterprise software inside SurveyMonkey on the way to its IPO, and I was inside Postmates, Guardicore, and Neuton.AI for their exits: $5B+ in combined value across the four. My consulting work has produced $20M+ in client revenue. Ordani, the app in these pages, is a HIPAA-compliant SaaS I built alone on the same AI tools this manual is about — hundreds of birth workers pay for it today.

PATH ONE · THE MANUAL

Nine more chapters: the spec, architecture, deploy day, security, Stripe, compliance, your first ten users, distribution, and when to hand off. Plus the companion files: prompt files, pre-flight checklists, spec templates.

micahjonesconsulting.com/playbook
Launch price $99, then $149.

PATH TWO · THE OPERATOR

Some builds need a second pair of hands, not another book. I take a small number of engagements: the strategy and the software, from one person. Thirty minutes, bring the problem.

micahjonesconsulting.com/book